Centered coefficient bitplanes

2026-11-05 · T-1604

The decoder now uses centered coefficient magnitudes and combines coefficient signs with the existing monomial-sign masks. The saved packet needs 128 population counts instead of 144, with the same 328-word payload and one final modular reduction. These counts are derived from source.

Saved packet: 144 ⟶ 128 popcounts; 328 payload words unchanged

Derived source counts; no elapsed-time comparison

Exact construction

For 0 <= c < r, take a = min(c, r-c) and sign negative exactly when c > floor(r/2). Then the signed magnitude equals c modulo r. XOR its sign into the parity mask. Support masks still annihilate monomials with zero inputs, including repeated even powers. Magnitude-bit population counts followed by the exact signed Horner fold recover the same polynomial for every ternary input.

Bounds and tradeoff

The supported modulus is at most 2^63-1, so magnitude planes need at most 62 bits. The retained 63-digit i128 bound covers each doubling and addition. For the saved packet, eight magnitude rows plus one sign row replace nine coefficient rows. For an arbitrary program whose magnitude bit length does not shrink, the new sign row adds one word per 64 terms. Constructor centering and sign-row loads belong in complete-cost accounting.

Checks

All 331 native tests pass with overflow checks enabled. New controls cover 25,704 small-residue/ternary cases and 105 midpoint/maximum cases. Retained checks include the full 479-input identity, word boundaries, exact wide-fold extremes and eight recovery fixtures. All 39 source/data hashes verify; recovery source is unchanged. Raw logs retain 23 compiler warnings. No timing comparison or cloud job ran.

Evidence and next task

See DERIVATION.md, control.json, source-lock.json and receipt.json. Source-lock generation and build observation overlapped; post-build hashes verify, without a claim about lock-completion ordering. The original 4563-case cohort remains separate. Next account for the complete decoder and setup; compact coefficient-program construction remains the structural dependency.

Prior arithmetic context: Umuroglu et al. (2019), sections 2.1–2.2. Remainder semantics: Rust i128 documentation. No novelty or CPU performance claim is imported from those sources.